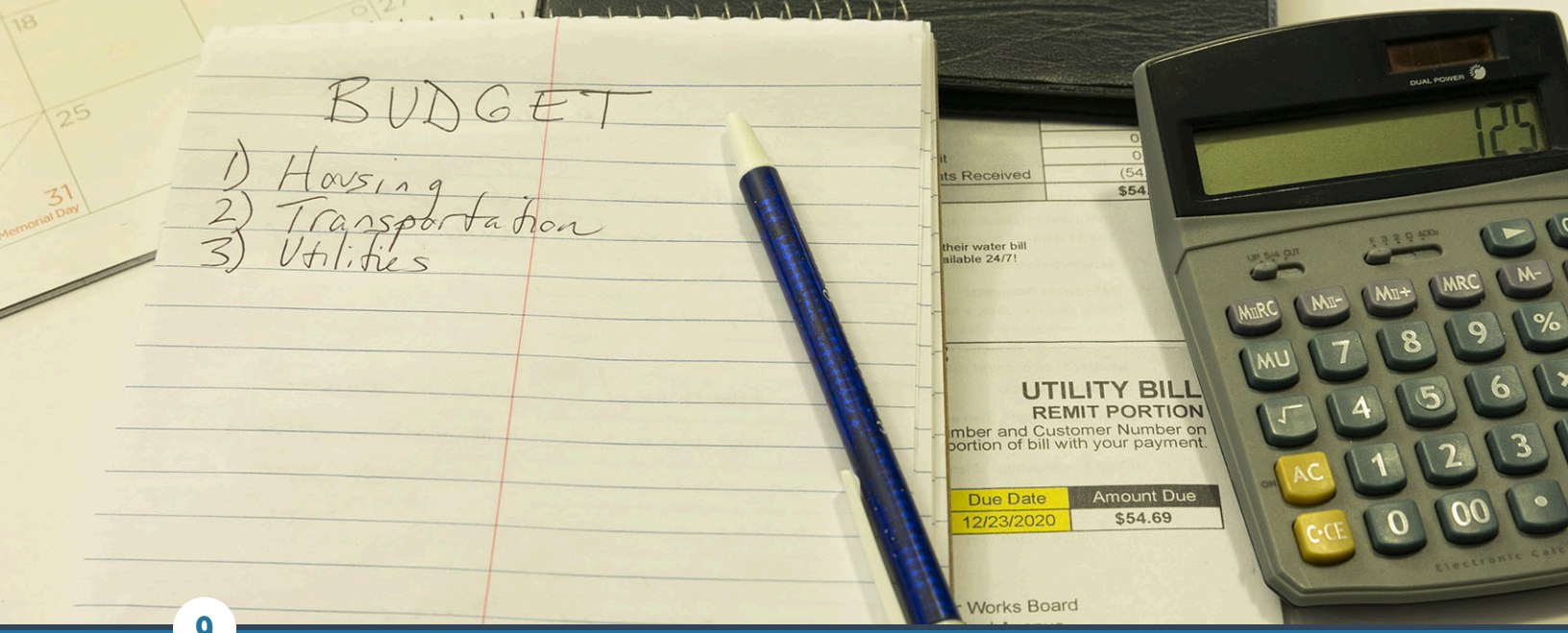


9 Lists

Figure 9.1 credit: modification of work "Budget and Bills" by Alabama Extension/Flickr, Public Domain

Chapter Outline

9.1 Modifying and iterating lists
9.2 Sorting and reversing lists
9.3 Common list operations
9.4 Nested lists
9.5 List comprehensions
9.6 Chapter summary

Introduction

Programmers often work on collections of data. Lists are a useful way of collecting data elements. Python lists are extremely flexible, and, unlike strings, a list's contents can be changed.

The Objects chapter introduced lists. This chapter explores operations that can be performed on lists.

9.1 Modifying and iterating lists

Learning objectives

By the end of this section you should be able to

- Modify a list using `append()`, `remove()`, and `pop()` list operations.
- Search a list using a `for` loop.

Using list operations to modify a list

An `append()` operation is used to add an element to the end of a list. In programming, **append** means add to the end. A `remove()` operation removes the specified element from a list. A `pop()` operation removes the last item of a list.

EXAMPLE 9.1

Simple operations to modify a list

The code below demonstrates simple operations for modifying a list.

Line 8 shows the `append()` operation, line 12 shows the `remove()` operation, and line 17 shows the `pop()` operation. Since the `pop()` operation removes the last element, no parameter is needed.

```
1  """Operations for adding and removing elements from a list."""
2
3  # Create a list of students working on a project
4  student_list = ["Jamie", "Vicky", "DeShawn", "Tae"]
5  print(student_list)
6
7  # Another student joins the project. The student must be added to
   the list.
8  student_list.append("Ming")
9  print(student_list)
10
11 # "Jamie" withdraws from the project. Jamie must be removed from
   the list.
12 student_list.remove("Jamie")
13 print(student_list)
14
15 # Suppose "Ming" had to be removed from the list.
16 # A pop() operation can be used since Ming is last in the list.
17 student_list.pop()
18 print(student_list)
```

The above code's output is:

```
['Jamie', 'Vicky', 'DeShawn', 'Tae']
['Jamie', 'Vicky', 'DeShawn', 'Tae', 'Ming']
['Vicky', 'DeShawn', 'Tae', 'Ming']
['Vicky', 'DeShawn', 'Tae']
```

CONCEPTS IN PRACTICE

Modifying lists

1. Which operation can be used to add an element to the end of a list?
   a. `add()`
   b. `append()`
   c. `pop()`
2. What is the correct syntax to remove the element 23 from a list called `number_list`?
   a. `remove()`
   b. `number_list.remove()`

c. `number_list.remove(23)`

3. Which operation can be used to remove an element from the end of a list?
   a. only `pop()`
   b. only `remove()`
   c. either `pop()` or `remove()`

Iterating lists

An iterative for loop can be used to iterate through a list. Alternatively, lists can be iterated using list indexes with a counting for loop. The animation below shows both ways of iterating a list.

CHECKPOINT

Using len() to get the length of a list

Using len() to get the length of a list; ch 9, video 1 (https://www.openstax.org/r/len-function)

CONCEPTS IN PRACTICE

Iterating lists

For the following questions, consider the list:

```
my_list = [2, 3, 5, 7, 9]
```

4. How many times will the following `for` loop execute?
   ```
   for element in my_list:
   ```
   a. 5
   b. 4

5. What is the final value of `i` for the following counting `for` loop?
   ```
   for i in range(0, len(my_list)):
   ```
   a. 9
   b. 4
   c. 5

6. What is the output of the code below?

   ```
   for i in range(0, len(my_list), 2):
     print(my_list[i], end=' ')
   ```

   a. `2 5 9`
   b. `2 3 5 7 9`
   c.
   ```
   2
   5
   9
   ```

TRY IT

Sports list

Create a list of sports played on a college campus. The sports to be included are baseball, football, tennis, and table tennis.

Next, add volleyball to the list.

Next, remove `"football"` from the list and add `"soccer"` to the list.

Show the list contents after each modification.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/9-1-modifying-and-iterating-lists)

TRY IT

Simple Searching

Write a program that prints `"found!"` if `"soccer"` is found in the given list.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/9-1-modifying-and-iterating-lists)

9.2 Sorting and reversing lists

Learning objectives

By the end of this section you should be able to

- Understand the concept of sorting.
- Use built-in `sort()` and `reverse()` methods.

Sorting

Ordering elements in a sequence is often useful. **Sorting** is the task of arranging elements in a sequence in ascending or descending order.

Sorting can work on numerical or non-numerical data. When ordering text, dictionary order is used. Ex: "bat" comes before "cat" because "b" comes before "c".

CHECKPOINT

Sorting

Sorting; ch 9, video 2 (https://www.openstax.org/r/sorting)

CONCEPTS IN PRACTICE

Sorting

1. What would be the last element of the following list if it is sorted in descending order?
   `[12, 3, 19, 25, 16, -3, 5]`
   a. `25`
   b. `-3`
   c. `5`

2. Arrange the following list in ascending order.
   `["cat", "bat", "dog", "coyote", "wolf"]`
   a. `["bat", "cat", "coyote", "dog", "wolf"]`
   b. `["wolf", "coyote", "dog", "cat", "bat"]`

3. How are the words `"flask"` and `"flash"` related in Python?
   a. `"flask" < "flash"`
   b. `"flask" == "flash"`
   c. `"flask" > "flash"`

Using `sort()` and `reverse()`

Python provides methods for arranging elements in a list.

- The `sort()` method arranges the elements of a list in ascending order. For strings, ASCII values are used and uppercase characters come before lowercase characters, leading to unexpected results. Ex: `"A"` is ordered before `"a"` in ascending order but so is `"G"`; thus, `"Gail"` comes before `"apple"`.
- The `reverse()` method reverses the elements in a list.

EXAMPLE 9.2

Sorting and reversing lists

```
# Setup a list of numbers
num_list = [38, 92, 23, 16]
print(num_list)

# Sort the list
num_list.sort()
print(num_list)

# Setup a list of words
dance_list = ["Stepping", "Ballet", "Salsa", "Kathak", "Hopak", "Flamenco", "Dabke"]

# Reverse the list
dance_list.reverse()
print(dance_list)
```

```
# Sort the list
dance_list.sort()
print(dance_list)
```

The above code's output is:

```
[38, 92, 23, 16]
[16, 23, 38, 92]
["Dabke", "Flamenco", "Hopak", "Kathak", "Salsa", "Ballet", "Stepping"]
["Ballet", "Dabke", "Flamenco", "Hopak", "Kathak", "Salsa", "Stepping"]
```

CONCEPTS IN PRACTICE

sort() and reverse() methods

Use the following list for the questions below.

```
board_games = ["go", "chess", "scrabble", "checkers"]
```

4. What is the correct way to sort the list `board_games` in ascending order?
   a. `sort(board_games)`
   b. `board_games.sort()`
   c. `board_games.sort('ascending')`
5. What is the correct way to reverse the list `board_games`?
   a. `board_games.reverse()`
   b. `reverse(board_games)`
6. What would be the last element of `board_games` after the `reverse()` method has been applied?
   a. `'go'`
   b. `'checkers'`
   c. `'scrabble'`

TRY IT

Sorting and reversing

Complete the program below to arrange and print the numbers in ascending and descending order.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/9-2-sorting-and-reversing-lists)

9.3 Common list operations

Learning objectives

By the end of this section you should be able to

- Use built-in functions `max()`, `min()`, and `sum()`.
- Demonstrate how to copy a list.

Using built-in operations

The `max()` function called on a list returns the largest element in the list. The `min()` function called on a list returns the smallest element in the list. The `max()` and `min()` functions work for lists as long as elements within the list are comparable.

The `sum()` function called on a list of numbers returns the sum of all elements in the list.

EXAMPLE 9.3

Common list operations

```
"""Common list operations."""

# Set up a list of number
snum_list = [28, 92, 17, 3, -5, 999, 1]

# Set up a list of words
city_list = ["New York", "Missoula", "Chicago", "Bozeman",
"Birmingham", "Austin", "Sacramento"]

# Usage of the max() funtion
print(max(num_list))

# max() function works for strings as well
print(max(city_list))

# Usage of the min() funtion which also works for strings
print(min(num_list))

print(min(city_list))

# sum() only works for a list of numbers
print(sum(num_list))
```

The above code's output is:

```
999
Sacramento
-5
Austin
1135
```

CONCEPTS IN PRACTICE

List operations

1. What is the correct way to get the minimum of a list named `nums_list`?
   a. `min(nums_list)`
   b. `nums_list.min()`
   c. `minimum(nums_list)`

2. What is the minimum of the following list?
   `["Lollapalooza", "Coachella", "Newport Jazz festival", "Hardly Strictly Bluegrass", "Austin City Limits"]`
   a. `Coachella`
   b. `Austin City Limits`
   c. The minimum doesn't exist.

3. What value does the function call return?
   `sum([1.2, 2.1, 3.2, 5.9])`
   a. `sum()` only works for integers.
   b. `11`
   c. `12.4`

Copying a list

The `copy()` method is used to create a copy of a list.

CHECKPOINT

Copying a list

Copying a list; ch 9, video 3 (https://www.openstax.org/r/copying-list)

CONCEPTS IN PRACTICE

Copying a list

4. What is the output of the following code?

```
my_list = [1, 2, 3]
list2 = my_list
list2[0] = 13
print(sum(my_list))
```

   a. `6`
   b. `13`
   c. `18`

5. What is the output of the following code?

```
my_list = [1, 2, 3]
```

```
list2 = my_list.copy()
list2[0] = 13
print(max(my_list))
```

a. 3
b. 13
c. 18

6. What is the output of the following code?

```
my_list = ["Cat", "Dog", "Hamster"]
list2 = my_list
list2[2] = "Pigeon"
print(sum(my_list))
```

a. CatDogPigeon
b. Error

TRY IT

Copy

Make a copy of `word_list` called wisdom. Sort the list called wisdom. Create a sentence using the words in each list and print those sentences (no need to add periods at the end of the sentences).

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/9-3-common-list-operations)

9.4 Nested lists

Learning objectives

By the end of this section you should be able to

- Demonstrate the use of a list-of-lists to structure data.
- Demonstrate individual element addressing using multi-dimensional indexing.
- Use nested loops to iterate a list-of-lists.

List-of-lists

Lists can be made of any type of element. A list element can also be a list. Ex: `[2, [3, 5], 17]` is a valid list with the list `[3, 5]` being the element at index 1.

When a list is an element inside a larger list, it is called a **nested list**. Nested lists are useful for expressing multidimensional data. When each of the elements of a larger list is a smaller list, the larger list is called a **list-of-lists**.

Ex: A table can be stored as a two-dimensional list-of-lists, where each row of data is a list in the list-of-lists.

CHECKPOINT

List-of-lists

List-of-lists; ch 9, video 4 (https://www.openstax.org/r/list-of-lists)

CONCEPTS IN PRACTICE

Lists

For each of the questions below, consider the following matrix:

$$\text{mat A} = \begin{bmatrix} 7 & 4 & 5 \\ 3 & 9 & 6 \\ 1 & 2 & 8 \end{bmatrix}$$

1. What would be the correct way to represent `matA` in Python?
   a. `[[7, 4, 5], [3, 9, 6], [1, 2, 8]]`
   b.
   ```
   [7, 4, 5
    3, 9, 6
    1, 2, 8]
   ```
   c. `[[7, 3, 1], [4, 9, 2], [1, 2, 8]`
2. What would be the correct index for the number `6` in the above list?
   a. `[5]`
   b. `[2][1]`
   c. `[1][2]`
3. What would be the result of the following code:

   ```
   print(matA[0])
   ```

   a. `Error`
   b. `7`
   c. `[7, 4, 5]`

Using nested loops to iterate nested lists

A nested loop structure can be used to iterate a list-of-lists. For a two-dimensional list-of-lists, an outer `for` loop can be used for rows, and an inner `for` loop can be used for columns.

EXAMPLE 9.4

Iterating a list-of-lists

The code below demonstrates how to iterate a list-of-lists.

The outer loop on line 9 goes element by element for the larger list. Each element in the larger list is a list. The inner loop on line 10 iterates through each element in each nested list.

```
1 """Iterating a list-of-lists."""
```

```
 2
 3  # Create a list of numbers
 4  list1 = [[1, 2, 3],
 5           [1, 4, 9],
 6           [1, 8, 27]]
 7
 8  # Iterating the list-of-lists
 9  for row in list1:
10    for num in row:
11      print(num, end=" ")
12    print()
```

The above code's output is:

```
1 2 3
1 4 9
1 8 27
```

CONCEPTS IN PRACTICE

Iterating a list-of-lists

For each question below, consider the following list:

```
my_list = [[7, 4, 5, 12],
      [24, 3, 9, 16],
      [12, 8, 91, -5]]
```

4. Which code prints each number in `my_list` starting from `7`, then `4`, and so on ending with `-5`?

   a.
   ```
   for row in my_list:
       for elem in row:
         print(elem)
   ```

   b.
   ```
   for elem in my_list:
         print(elem)
   ```

5. The `range()` function can also be used to iterate a list-of-lists. Which code prints each number in `my_list` starting from `7`, then `4`, and so on, ending with `-5`, using counting `for` loops?

   a.
   ```
   for column_index in range(0, len(my_list[0])):
       for row_index in range (0, len(my_list)):
         print(my_list[row_index][column_index])
   ```

   b.
   ```
   for row_index in range(0, len(my_list)):
       for column_index in range (0, len(my_list)):
         print(my_list[row_index][column_index])
       print()
   ```

   c.
   ```
   for row_index in range(0, len(my_list)):
       for column_index in range (0, len(my_list[0])):
         print(my_list[row_index][column_index])
   ```

TRY IT

Matrix multiplication

Write a program that calculates the matrix multiplication product of the matrices matW and matZ below and prints the result. The expected result is shown.

$$\text{mat W} = \begin{bmatrix} 13 & 4 & 5 \\ 2 & -9 & 7 \\ 7 & 3 & 19 \end{bmatrix} \text{mat Z} = \begin{bmatrix} 2 & 1 & 5 \\ 3 & 7 & 9 \\ -1 & 13 & 19 \end{bmatrix}$$

$$\text{result} = \begin{bmatrix} 33 & 106 & 196 \\ -30 & 30 & 62 \\ 4 & 275 & 423 \end{bmatrix}$$

In the result matrix, each element is calculated according to the position of the element. The result at position `[i][j]` is calculated using row `i` from the first matrix, W, and column `j` from the second matrix, Z.

Ex:

`result[1][2]` = (row 1 in W) times (column 2 in Z)

$$\text{result[1][2]} = \begin{bmatrix} 2 & -9 & 7 \end{bmatrix} * \begin{bmatrix} 5 \\ 9 \\ 19 \end{bmatrix}$$

$$\text{result[1][2]} = 2 * 5 + (-9) * 9 + 7 * 19 = 10 - 81 + 133 = 62$$

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/9-4-nested-lists)

9.5 List comprehensions

Learning objectives

By the end of this section you should be able to

- Identify the different components of a list comprehension statement.
- Implement filtering using list comprehension.

List comprehensions

A **list comprehension** is a Python statement to compactly create a new list using a pattern.

The general form of a list comprehension statement is shown below.

```
list_name = [expression for loop_variable in iterable]
```

`list_name` refers to the name of a new list, which can be anything, and the `for` is the `for` loop keyword. An expression defines what will become part of the new list. `loop_variable` is an iterator, and `iterable` is an object that can be iterated, such as a list or string.

EXAMPLE 9.5

Creating a new list with a list comprehension

A list comprehension shown below in the second code has the same effect as the regular `for` loop shown in the first code. The resultant list is `[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]` in both cases.

Creating a list of squares using a `for` loop.

```
# Create an empty List.
squares_list = []

# Add items to a list, as squares of numbers starting at 0 and ending at 9.
for i in range(10):
  squares_list.append(i*i)
```

Creating a list of squares using the list comprehension.

```
square_list = [i*i for i in range(10)]
```

The expression `i*i` is applied for each value of the `loop_variable` `i`.

EXAMPLE 9.6

A Dr. Seuss poem

A list comprehension can be used to create a list based on another list. In line 6, the `for` loop is written on the list `poem_lines`.

```
1 # Create a list of words
2 words_list = ["one", "two", "red", "blue"]
3
4 # Use a list comprehension to create a new list called poem_lines
5 # Inserting the word "fish" attached to each word in words_list
6 poem_lines = [w + " fish" for w in words_list]
7 for line in poem_lines:
8   print(line)
```

The above code's output is:

```
one fish
two fish
red fish
blue fish
```

CONCEPTS IN PRACTICE

List comprehensions

1. The component of a list comprehension defining an element of the new list is the _____.
   a. expression
   b. loop_variable
   c. container

2. What would be the contents of `b_list` after executing the code below?

```
a_list = [1, 2, 3, 4, 5]
b_list = [i+2 for i in a_list]
```

   a. `[1, 2, 3, 4, 5]`
   b. `[0, 1, 2, 3, 4]`
   c. `[3, 4, 5, 6, 7]`

3. What does `new_list` contain after executing the statement below?

```
new_list = [i//3 for i in range(1, 15, 3)]
```

   a. `[0.3333333333333333, 1.3333333333333333, 2.3333333333333335, 3.3333333333333335, 4.333333333333333]`
   b. `[0, 1, 2, 3, 4]`
   c. `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]`

Filtering using list comprehensions

List comprehensions can be used to filter items from a given list. A *condition* is added to the list comprehension.

```
list_name = [expression for loop_variable in container if condition]
```

In a filter list comprehension, an element is added into `list_name` only if the condition is met.

CHECKPOINT

Filtering a list

Filtering a list; ch 9, video 5 (https://www.openstax.org/r/filtering-list)

CONCEPTS IN PRACTICE

Filtering using list comprehensions

For each code using list comprehension, select the correct resultant list in `new_list`.

4. `my_list = [21, -1, 50, -9, 300, -50, 2]`

```
new_list = [m for m in my_list if m < 0]
```

a. [21, 50, 300, 2]
b. [21, -1, 50, -9, 300, -50, 2]
c. [-1, -9, -50]

5.
```
my_string = "This is a home."

new_list = [i for i in my_string if i in 'aeiou']
```

a. [i, i, a, o, e]
b. ['i', 'i'', 'a', 'o', 'e']
c. Error

6.
```
new_list = [r for r in range (0, 21, 2) if r%2 != 0]
```

a. []
b. [21]
c. [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]

TRY IT

Selecting five-letter words

Write a program that creates a list of only five-letter words from the given list and prints the new list.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/9-5-list-comprehensions)

TRY IT

Books starting with "A"

Write a program that selects words that begin with an "A" in the given list. Make sure the new list is then sorted in dictionary order. Finally, print the new sorted list.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/9-5-list-comprehensions)

9.6 Chapter summary

Highlights from this chapter include:

- Lists are mutable and can be easily modified by using `append()`, `remove()`, and `pop()` operations.
- Lists are iterable and can be iterated using an iterator or element indexes.
- The `sort()` operation arranges the elements of a list in ascending order if all elements of the list are of the same type.

- The `reverse()` operation reverses a list.
- The `copy()` method is used to create a copy of a list.
- Lists have built-in functions for finding the maximum, minimum, and summation of a list for lists with only numeric values.
- Lists can be nested to represent multidimensional data.
- A list comprehension is a compact way of creating a new list, which can be used to filter items from an existing list.

At this point, you should be able to write programs using lists.

Function	Description
`append(element)`	Adds the specified element to the end of a list.
`remove(element)`	Removes the specified element from the list if the element exists.
`pop()`	Removes the last element of a list.
`max(list)`	Returns the maximum element of the list specified.
`min(list)`	Returns the maximum element of the list specified.
`sum(list)`	Returns the summation of a list composed of numbers.
`sort()`	Sorts a list on which the method is called in ascending order.
`reverse()`	Reverses the order of elements in a list.
`copy()`	Makes a complete copy of a list.

Table 9.1 Chapter 9 reference.